

Humor Detection and Explainability via RoBERTa

Brendan George Lauterborn
Department of Computer Science
Towson University
Towson, United States
blaute3@students.towson.edu

Abstract—Humor is important in everyday human interactions and allows us to connect, communicate, and share feelings with others. Without humor, we would not be the unique species that we are today. Since humor often relies on context, ambiguity, and wordplay, it is difficult to build a machine learning model that recognizes humor from natural language alone. This project focuses on building a binary classification model that determines whether a set of text is humorous or non-humorous. We fine-tune a pre-trained Large Language Model (LLM) known as RoBERTa to help us with this task. On top of that, we explore Explainable AI (XAI) to help showcase which individual tokens led to the decision by the model. The goal of this work is to understand the complexity of human humor and to develop effective natural language processing techniques for understanding humor.

I. INTRODUCTION

Humans share humor not only in person, but online as well. There are many popular entertainment applications that allow users to post, comment, and interact with other users. Some applications include Instagram, Twitter, TikTok, and Reddit. These interactions help to drive entertainment, social connection, and community engagement. Additionally, many modern applications are now including chatbot systems that allow users to ask questions, receive recommendations, and interact with AI systems. Thus, it is clear that humor is a fundamental aspect of not only human-to-human interaction, but also human-computer interaction. However, building a machine learning model that understands humor is a difficult task.

This remains a difficult task in natural language processing (NLP) because humor often depends on context, cultural references, sarcasm, irony, exaggeration, and wordplay. Unlike traditional text classification problems, humor is highly subjective and varies between individuals. Therefore, building an accurate machine learning model that correctly classifies humor from non-humor is difficult.

NLP techniques have dramatically improved over the last century. The early models started as simple statistical models and transformed into rule-based models. From here, the models then transformed into neural networks and finally to modern transformers. These transformers are able to capture deep semantic meaning and latent features from text. Dating all the way back to 1913 was N-Grams by Andrey Markov. Following this, there was Bag-of-Words (BoW) in 1954. These models focused on word frequency and local text patterns. As computational power increased and research grew, Recurrent Neural Nets (RNN) were developed in 1980. Following this,

Long-Short-Term-Memory (LSTM) was introduced in 1997. These techniques allowed models to better capture meaning from text. Following this, GloVe and Encoder-Decoder were introduced in 2014 where major breakthroughs were made. Encoder-Decoder further improved the model’s ability to capture semantic meaning from text due to the introduction of the attention mechanism. Previous models struggled with large sequences because the input sequence was compressed to a fixed vector length. The attention mechanism addressed this issue by determining which words in a sequence are most important when processing. A relative importance score is encoded and assigned to each word while processing the sequence. This alignment score is fed to a softmax function which generates normalized weights. Essentially, this allows the model to focus on the most relevant words in the sequence.

Finally, Transformers were introduced in 2017. Transformers built upon encoder-decoders from previous years. They are considered the largest breakthrough in NLP tasks due to the elimination of recurrence or convolutions in its architecture. This means that each word is processed in parallel, rather than sequentially. The transformer architecture is what drives powerful models like ChatGPT.

Bidirectional Encoder Representations from Transformers (BERT) was originally introduced in 2018 by Devlin et al. BERT introduced Bidirectional Encoder Representations from Transformers, which allows the model to learn and understand bidirectional contextual relationships within text. This model achieved state-of-the-art results and performance for NLP tasks. RoBERTa is an adaptation of the BERT model that shares the same structure, but introduces improvements to the training steps of the model. Due to RoBERTa’s strong contextual understanding and ability to capture semantic relationships between words, it is well-suited for our humor detection task. Humor often depends on ambiguity, sarcasm, irony, and wordplay. Therefore, RoBERTa is used in this project to classify text as humorous or non-humorous. In addition, we briefly explore XAI techniques to help us understand the importance of each token within the sequence. More specifically, we focus on Integrated Gradients to provide insight into how transformer-based models interpret humorous text.

II. DATASET DESCRIPTION

For this study, we used the *200k Short Texts for Humor Detection* obtained from Kaggle. The dataset is 14.87 MB in size and contains 200,000 instances with 2 attributes. The

attributes are *text* and *humor*, where text represents the natural language and humor is the label that determines humorous or non-humorous. True represents the associated text being humorous, while False represents the associated text being non-humorous. There is an exact 50/50 split of humorous to non-humorous text within the dataset, so we do not have to worry about any class imbalance.

III. RELATED WORKS

In Humour Classification by Fine-tuning LLMs: CYUT at CLEF 2024 JOKER Lab Subtask Humour Classification According to Genre and Technique, Wu et al. investigated humor classification by fine-tuning LLMs such as RoBERTa, GPT-4, and Llama 3-8B. Their work separated the jokes into the following classes: irony, sarcasm, exaggeration, incongruity-absurdity, self-deprecating, and wit-surprise. When fine-tuning the Llama model, they followed the the Stanford Alpaca Format. In this format, the model is told the instructions, “Classify the following text into one of the classes” [1]. Next, the model is provided the six classification classes and their descriptions.

Based on their results from RoBERTa, GPT-4, the authors discovered that the models struggled to correctly identify irony and sarcasm from text. To combat this, the prompt was modified to encourage the model to focus more on identifying irony and sarcasm. The authors also applied Chain-of-Thought (CoT) prompt engineering to attempt to improve performance. The CoT phrase that was added encouraged the model to “think step by step” before coming to a conclusion [1]. The work conducted by the authors demonstrates the difficulty of humor classification and the importance of prompting strategies in humor detection for LLMs.

In The Improved Neural Network Model in Humor Detection with Traditional Humor Theory, Sun et al. explored humor detection using RoBERTa combined with traditional humor theories such as incongruity theory and humor anchor extraction [2]. The incongruity theory indicates that, “humor is produced because certain things or events that we perceive or think violate our standard thinking patterns and normal expectations” [2]. Sun et al. also incorporated the concept of Humor Anchors into their RoBERTa-based humor detection framework [2]. Humor Anchors refer to important words within a sentence that contribute significantly to the humorous effect. Their work suggested that removing these key contextual components can reduce humor within a sentence.

IV. EXPLORATORY DATA ANALYSIS

Looking at Fig. 1, we are able to understand the distribution of the length of the texts within our dataset. It is clear that the length follows a normal distribution centered around 65 characters. The largest text length is around 98 characters. This figure helped with selecting the maximum sequence length during tokenization.

Table I shows the average character length of non-humorous and humorous texts. A two-sample t-test showed that the

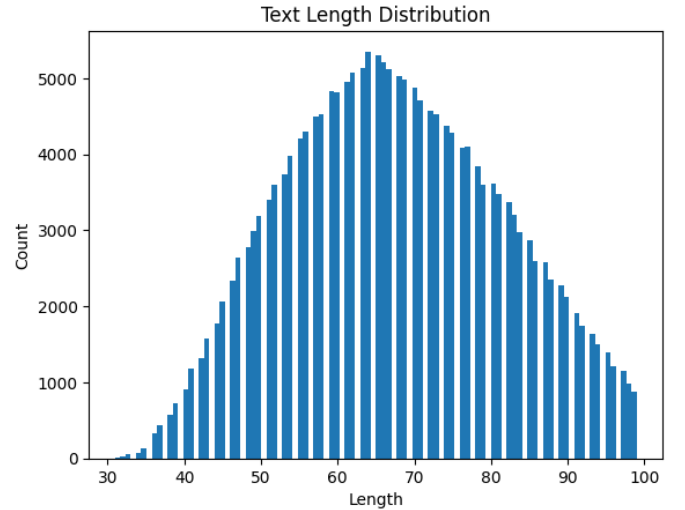


Fig. 1: Length of texts within the Dataset

difference in character length between humorous and non-humorous samples was statistically significant ($t = -80.90$, $p < 0.001$). Humorous texts had more characters than non-humorous texts. However, this alone does not predict humor.

TABLE I: Average Text Length by Class

Class	Average Length
Non-Humorous (0)	64.95
Humorous (1)	69.99

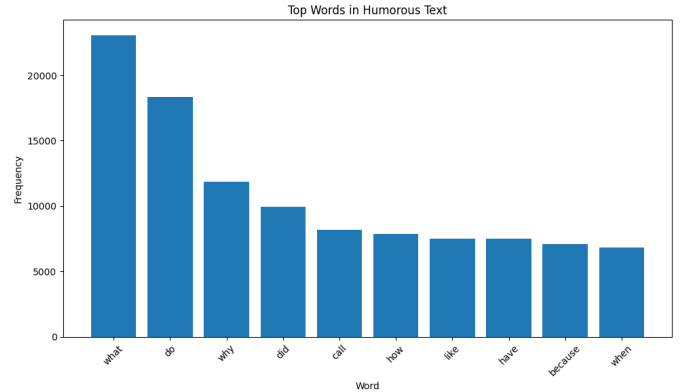


Fig. 2: Most Common Humorous Words

Looking at Fig. 2 and Fig. 3, we see the most common words for each class. In Fig. 2, the humorous class seems to contain question oriented words. This may suggest that humorous text may rely on setup or rhetorical questioning to achieve humor. In Fig. 3, the most common words seem to revolve around a political figure, in this case it is words like *Trump* and *Donald*. There are still many question oriented words within the non-humorous text as well.

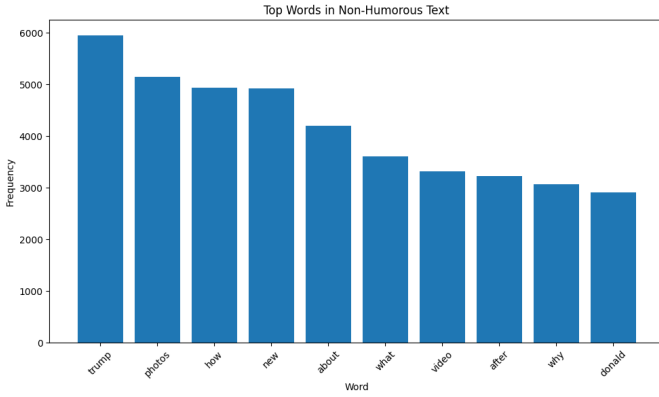


Fig. 3: Most Common Non-Humorous Words

V. MODEL DESCRIPTION

Bidirectional Encoder Representations from Transformers (BERT) is designed to learn latent features of text to understand meaning and context. This is achieved by using a bidirectional transformer, which allows BERT to understand a sentence both forward and backward. This design enables BERT to capture richer semantics of text more effectively than previous models that only process the text left-to-right. BERT achieves this by training on two tasks simultaneously. These tasks are Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).

In MLM, the input text is first tokenized using WordPiece. The text is also wrapped with special tokens. [CLS] is placed at the beginning and [SEP] is placed between sentences and at the end. In [5], we see that 15% of the tokens are chosen to be masked. 80% of the masked tokens are replaced with [MASK]. To combat overfitting, 10% of the chosen tokens are replaced with a random token. The remaining 10% are left as is. This sequence is then fed to BERT where it goes through the transformer layers. BERT attempts to predict the original tokens that were masked. By doing this, BERT is forced to understand the bidirectional context of the text and therefore learns the deep semantic features of the text.

The next step to BERT is NSP. NSP is essentially a binary classification problem. Two sentences are paired together and fed to the transformer encoder. 50% of the time Sentence B actually follows sentence A. In the remaining pairs, sentence B is a random sentence that does not follow sentence A. BERT predicts whether sentence B is the real subsequent sentence or not. From this training, BERT learns the context across different sentences. The combination of MLM and NSP allows BERT to understand deeper semantics between sentences and text. After pre-training BERT, the model is adapted to a classification task by replacing the output layer and fine-tuning it on a dataset.

RoBERTa is an adaptation of the BERT model that shares the same structure, but introduces improvements to the training steps of the model as discussed in [5]. In RoBERTa, NSP is removed completely. Studies showed that NSP contributed very little to the performance of the model and even introduced

noise. RoBERTa focuses on the MLM step of the training process instead. On top of that, rather than using a static masking strategy, RoBERTa uses a dynamic masking strategy to mask the tokens. In BERT, the positions of the masked tokens were fixed. The RoBERTa model improves this strategy by randomly sampling the masked positions at each training pass. This allows the model to encounter more diverse patterns and ultimately understand the semantics of the text at a deeper level. During the training phase for BERT, batch sizes of 256 sequences were used. In RoBERTa, the batch sizes are increased to 8000 and the training duration is also extended. Additionally, RoBERTa is trained using roughly 160 GB of text, where BERT is only trained using 16 GB of text.

These improvements allow RoBERTa to capture deeper semantic meaning and achieve better results for NLP tasks than the baseline model BERT. Thus, we select RoBERTa as our pre-trained model to classify humor within text.

VI. METHODOLOGY

A. Experimental Setup

The experimental setup was implemented in Python using the PyTorch deep learning framework and the Hugging Face Transformers library. All model training and testing was conducted using Google Colab with access to their NVIDIA T4 GPU. When available, we accelerated the training and testing using CUDA to improve efficiency. The data was split 70/30, where 70% of the data is allocated to training the model, 15% is used to validate the model, and 15% is used to test the model.

B. Pre-processing

In order to preserve as much contextual meaning as possible, we did not remove and stop words, punctuation, or convert the text to lowercase. RoBERTa relies heavily on the semantic relationships between words to learn and interpret context from text.

RoBERTa requires us to pre-process our data to follow a standardized transformer format. This format requires our data to be transformed into two tensors, *input_ids* and *attention_mask*. Where,

- **input_ids**: An integer tensor that represents the tokenized text. Each token is mapped to a unique integer value based on the RoBERTa tokenizer vocabulary.
- **attention_mask**: A binary tensor that distinguishes between valid tokens and padded tokens within the sequence. For the RoBERTa tokenizer, a 0 indicates a padded token while a 1 represents a valid token that the model must attend to.

The longest sequence in our dataset was about 98 characters, making truncation unnecessary for this step. Our dataset was tokenized using the RoBERTa tokenizer with a max sequence length of 128 to be safe. All sequences will be padded in order to match the sequence length and ensure proper dimension. Once done properly, our text should follow,

'text': "My family tree is a cactus, we're all pricks.",

'input_ids': [0, 574, 108, 43434, ..., 2, 1, 1, 1,..., 1],
 'attention_mask': [1, 1, 1, ..., 0, 0, 0,..., 0]

where,

- **input_id = 0**: Beginning-of-sequence token (<s>)
- **input_id = 2**: End-of-sequence token (</s>)
- **input_id = 1**: Padding token (<pad>)
- **attention_mask = 0**: Padding token
- **attention_mask = 1**: Valid token

VII. MODEL PARAMETERS

A. Experimental Setup

We chose 5 epochs for this experiment to try and allow the model to properly train and understand the text without over training. However, 3 epochs would have been more than enough. During training, we only saved the model at each epoch if validation loss has decreased.

- **Model**: RoBERTa-base
- **Epochs**: 5
- **Learning Rate**: 2×10^{-5}
- **Optimizer**: AdamW
- **Maximum Sequence Length**: 128
- **Batch Size**: 32
- **Loss Function**: Cross Entropy Loss
- **Number of Labels**: 2 (Humorous and Non-Humorous)
- **Hardware**: NVIDIA T4 GPU using Google Colab with CUDA acceleration

VIII. RESULTS

Prior to fine-tuning our model, we first evaluate the RoBERTa base model against the testing dataset. This baseline model achieved a test loss: 0.690812 and a test accuracy of 56.11%. From Table II, we see that the model achieved a perfect recall for the “Not Humor” class. However, it produced very low recall for the “Humor” class, indicating that the model frequently predicted non-humorous labels. The baseline model achieved a near perfect precision score for the humor class. This shows that when the model does predict humor, it is almost always correct. However, this model predicts non-humorous most of the time. This demonstrates that the pretrained language model was not well adapted for humor detection before task-specific fine-tuning.

TABLE II: Classification Report for Pretrained RoBERTa Baseline

Class	Precision	Recall	F1-Score	Support
Not Humor	0.53	1.00	0.69	15000
Humor	0.99	0.12	0.22	15000

After training our model we tested it against our testing dataset. The results are shown in Table III. To evaluate our models, we use accuracy, precision, recall, and F1 score. Accuracy measures the amount of correct predictions to total predictions. The F1 score uses precision and recall. Precision is the amount of true positives over the sum of true positives and false positives. It measures the accuracy of positive

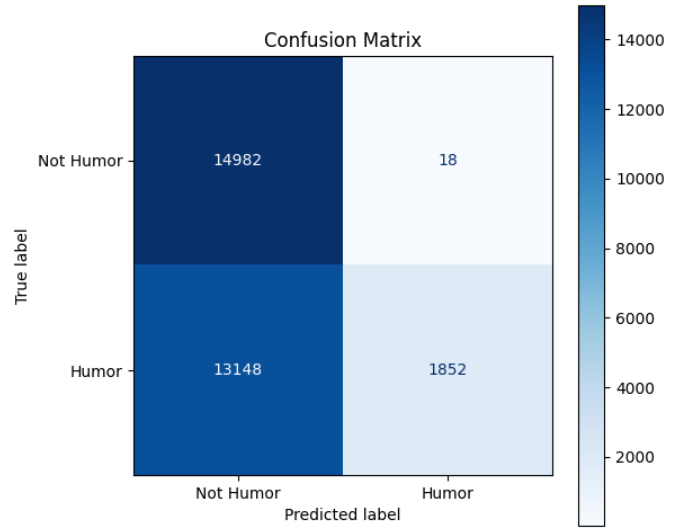


Fig. 4: Confusion Matrix - Baseline RoBERTa Model

predictions. Recall is the amount of true positives over the sum of true positives and false negatives. Essentially, the ratio of true positives to the actual amount of total positives. The F1 score is a good evaluation because it combines the mean of both precision and recall. If one measure is bad, then that greatly impacts the score.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

TABLE III: Classification Report for Humor Detection Model

Class	Precision	Recall	F1-Score	Support
Not Humor	0.97	0.99	0.98	15000
Humor	0.99	0.97	0.98	15000

Our model achieved very strong results, achieving a test loss of 0.043550 and a test accuracy of 98.23%. Overall, the results show that our model is very accurate. From Fig. 5, we observe that our model produces more false negatives than false positives. Specifically, our model was more likely to classify texts as non-humorous when the text is humorous. Our model accurately predicts almost all non-humorous text correctly and sometimes misclassified humorous text. Thus, the recall score for humorous texts is lower than the recall score for non-humorous texts. Looking at the precision scores, we see that the humorous class achieved a very high precision score. This means that when our model predicts a text as humorous, it is almost always correct. Specifically, the model

produces very few false positive predictions, meaning that non-humorous text is rarely classified as humorous. The F1-scores for both classes were approximately 0.98, indicating a strong balance between precision and recall. Since the F1-score considers both false positives and false negatives, the high scores suggest that the model performs consistently well across both classes.

Overall, our model performs extremely well. The model results in more false negatives than false positives. From this, we know that our model will misclassify humorous texts more often than it mislabels non-humorous text as humorous.

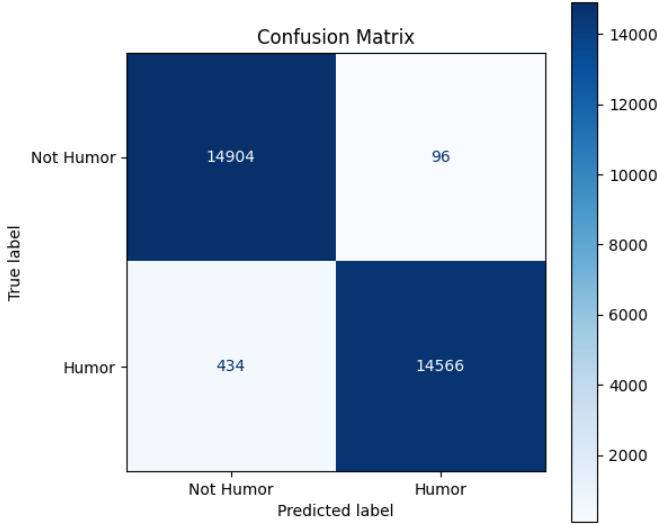


Fig. 5: Confusion Matrix - Humor Model

IX. EXPLAINABLE AI

To improve the interpretability of the model, we applied Integrated Gradients (IG) using the Captum library to analyze how each token contributes to the overall prediction of the model. The first step of the process is to create the baseline input, which consists entirely of padded tokens. This baseline sequence will be compared to the actual input sequence. Integrated Gradients repeatedly performs forward passes between the baseline and the real input while computing gradients with respect to the embedding layer. In other words, Integrated Gradients measures how the prediction changes as the input slowly moves from the blank baseline to the real sentence. These gradients measure how sensitive the model's prediction is to changes in each token.

As the input gradually transitions from the baseline to the actual sentence, gradients are repeatedly computed and accumulated to produce attribution scores for every token. Each token receives a score indicating how much it contributes to the final prediction. Every token embedding has many dimensions, so the attribution values are summed together to create one score per token. Finally, the padding tokens are removed so that only the actual tokens remain. The final tokens are visualized to highlight the most important tokens in determining whether a sentence is humorous [6].

This approach allows us to identify key tokens responsible for humor classification decisions and provides insight into how the model interprets contextual and semantic information within the text.

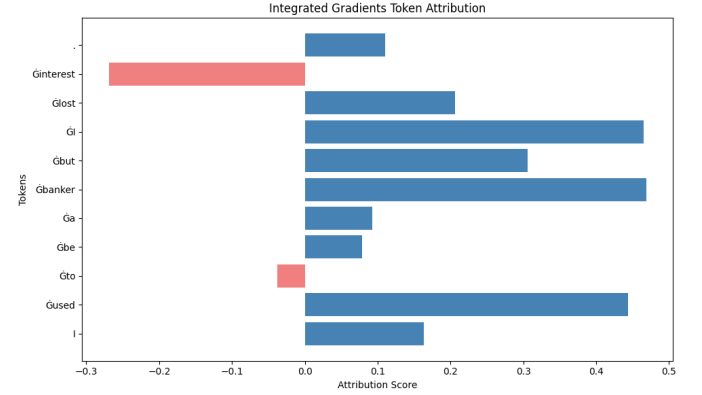


Fig. 6: IG Token Attribution

The Integrated Gradients visualization in Fig. 6 we tested the model against the sentence,

- **I used to be a banker but I lost interest.**

The model correctly classified the sentence as humorous. In addition, we observe that certain tokens contributed differently to the final prediction. For example, the token “interest” produced a negative attribution score, indicating that it pushed the prediction away from the humorous class. This is interesting because it goes against our initial belief that this token might represent the pun or the part that makes the sentence funny. This may occur because the word “interest” frequently appears in non-humorous financial or informational contexts within natural language. However, the surrounding tokens such as “banker”, “used”, “but”, “I”, and “lost” produced strong positive attribution scores. This suggests that the contextual interaction between multiple tokens influences the model's prediction rather than a single word in isolation. This directly demonstrates how transformer-based models such as RoBERTa rely heavily on contextual semantic relationships when interpreting humor.

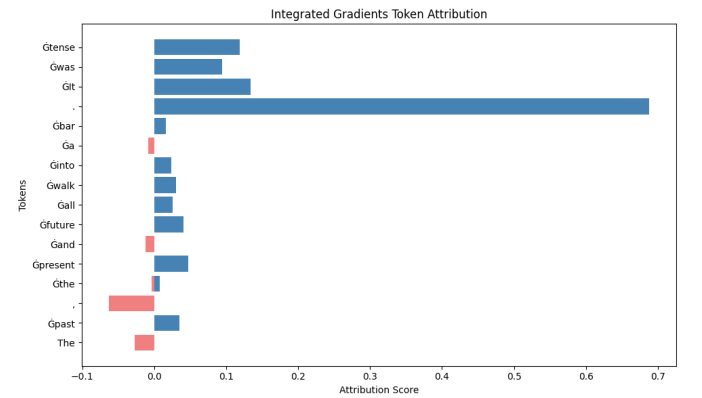


Fig. 7: IG Token Attribution

In Fig. 7, we test the model against the sentence,

- **The past, the present and the future all walk into a bar. It was tense**

The model correctly classified the sentence as humorous. In addition, the token attribution for the abrupt period in the middle of the sequence is very large. The words following the period, “It“, “was“, and “tense“, show a strong positive contribution to the model’s prediction. This suggests that the model is not only relying on semantic meaning, but also on structural cues and formatting when identifying humor from the sequences. These abrupt stops help to emphasize the delivery of certain jokes that are easier to understand when spoken out loud rather than in text.

X. DEPLOYMENT

In order to showcase our work, we built a simple full-stack architecture. The backend was developed using FastAPI and the frontend was built using React. The backend loaded our fine-tuned RoBERTa model and provided explainable AI functionality using Integrated Gradients from the Captum library to generate the token attribution scores. The frontend was developed using React and Recharts to provide an interactive user interface where users can input their text, receive humor predictions with confidence scores, and visualize the token attribution graphs. The backend environment was containerized using Docker, allowing the application to be deployed consistently across different systems without manually installing Python dependencies.

XI. LIMITATIONS AND FUTURE WORKS

The limitations of this project include the dataset. It was very difficult to find a proper dataset built around humor detection. Datasets that I read about from other papers were private and we were unable to access them. Our dataset contained very short texts that seemed to be straightforward for humans to identify as humor. However, building a model that can think like humans proved to be challenging.

The model occasionally assigned humorous classifications to non-humorous declarative statements. Integrated Gradients analysis revealed that punctuation tokens sometimes contributed heavily to the prediction, suggesting that the model may overweigh punctuation or structural cues. More specifically, this was observed when a period appears at the end of the sequence. To help solve this issue, the dataset should include more punctuation to non-humorous instances.

In the future, I would like build my own dataset by manually acquiring posts from multiple sources like X, Reddit, and TikTok comments. This would allow me to build a stronger model that

REFERENCES

- [1] Wu, Y., Chen, Y., Yu, C., & Tsai, Y. (2024). *Humour Classification by Fine-tuning LLMs: CYUT at CLEF 2024 JOKER Lab Subtask Humour Classification According to Genre and Technique*. CEUR Workshop Proceedings, Vol. 3740. Available: <https://ceur-ws.org/Vol-3740/paper-183.pdf>
- [2] Chen, G., & Soo, V.-W. (2021). *The Improved Neural Network Model in Humor Detection with Traditional Humor Theory*. IEEE Access. Available: <https://ieeexplore.ieee.org/document/9446017>
- [3] Raeini, M. G. (2025). *The evolution of language models: From N-Grams to LLMs, and beyond*. Natural Language Processing Journal, 12, 100168. Available: <https://www.sciencedirect.com/science/article/pii/S2949719125000445>
- [4] DataCamp. (2024). *How Transformers Work: A Detailed Exploration of Transformer Architecture*. Available: <https://www.datacamp.com/tutorial/how-transformers-work>
- [5] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Available: <https://arxiv.org/abs/1810.04805>
- [6] Wang, Y., Zhang, T., Guo, X., & Shen, Z. (2024). *Gradient based Feature Attribution in Explainable AI: A Technical Review*. Available: <https://arxiv.org/html/2403.10415v1>
- [7] Erdem, K. (2022). *XAI Methods — Integrated Gradients*. Available: <https://medium.com/@kemalpiro/xai-methods-integrated-gradients-6ee1fe4120d8>